

集成

从 OpenAI 迁移到 Kimi API

📄 复制页面



关于 API 兼容性

Kimi API 兼容了 OpenAI 的接口规范，你可以使用 OpenAI 提供的 [Python](#) 或 [NodeJS](#) SDK 来调用和使用 Kimi 大模型，这意味着如果你的应用和服务基于 openai 的模型进行开发，那么只需要将 `base_url` 和 `api_key` 替换成 Kimi 大模型的配置，即可无缝将你的应用和服务迁移至使用 Kimi 大模型，代码示例如下：

python node.js

```
from openai import OpenAI

client = OpenAI(
    api_key="MOONSHOT_API_KEY", # <--在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台获取的 API Key
    base_url="https://api.moonshot.cn/v1", # <-- 将 base_url 从 https://api.openai.com/v1 替换为 https://api.moonshot.cn/v1
)
```

我们会尽力保证 Kimi API 与 OpenAI 的兼容性，但在某些特殊场合，Kimi API 与 OpenAI 仍然存在一些差异和不同（但这不影响整体兼容性），我们将详细阐述 Kimi API 与 OpenAI 的不同点，并提出可行的迁移解决方案以帮助开发者顺利完成迁移工作。

以下是与 OpenAI 兼容的接口列表：

- `/v1/chat/completions`

- `/v1/files/{file_id}`

KIMI 开放平台

- `/v1/files/{file_id}/content`



>

temperature 和 N 值

当你使用 OpenAI 的接口时，你可以同时设置 `temperature=0` 和 `n>1`，即在 `temperature` 值为 0 的场合，同时返回多个不同的回答（即 `choices`）。

然而在 Kimi API 中，当你将 `temperature` 的值设置为 0 或接近 0 时（例如 0.001），我们将只能提供 1 个回答（即 `len(choices)=1`，如果你在把 `temperature` 设置为 0 的同时，使用了一个大于 1 的 `n` 值，我们将返回一个“非法请求”错误，即 `invalid_request_error`。

额外的，请注意：Kimi API 的 `temperature` 参数的取值范围是 `[0, 1]`，而 OpenAI 的 `temperature` 参数的取值范围是 `[0, 2]`。

迁移建议：

对于 `kimi-k2.6` 和 `kimi-k2.5` 模型，`temperature` 参数有特殊要求：

- 思考模式下固定使用 `temperature=1.0`
- 非思考模式下使用 `temperature=0.6`

如果指定其他值，将会报错。建议调用这些模型时不要显式设置 `temperature`，或按照上述要求设置。

stream 模式下的 usage 值

当你使用 OpenAI 的 `chat.completions` 接口时，在流式输出（即 `stream=True`）的场合下，输出结果默认不包含 `usage` 用量信息（包括 `prompt_tokens` / `completion_tokens` / `total_tokens`），OpenAI 提供了一个额外的参数 `stream_options={"include_usage": True}` 来使返回的最后一个数据块包含 `usage` 信息。

在 Kimi API 中，我们除了 `stream_options={"include_usage": True}` 参数外，还会在每个 `choice` 的结束数据块中放置 `usage` 信息（包括

迁移建议：通常情况下，开发者不需要做任何额外的兼容性举措，如果你的业务场景需要统计每个 choice 各自的 `usage` 信息，可以访问 `choice.usage` 字段，注意：在不同的 choices 中，仅有 `usage.completion_tokens` 和 `usage.total_tokens` 字段的值是不同的，choices 们拥有相同的 `usage.prompt_tokens` 值。

已被废弃的 `function_call`

OpenAI 在 2023 年提供了 `functions` 参数以开启函数调用（即 `function_call`）功能。经过功能迭代，OpenAI 后续推出了工具调用（即 `tool_calls`）功能，并将 `functions` 参数标记为已废弃（deprecated），这意味着在后续的 API 迭代中，`functions` 参数随时可能被移除。

Kimi API 完整支持了工具调用（即 `tool_calls`）的能力，同时，由于 `functions` 已被废弃，**Kimi API 不支持使用 `functions` 参数执行函数调用。**

迁移建议：如果你的应用或服务依赖于工具调用（即 `tool_calls`），那么不需要做任何额外的兼容性举措；如果你的应用或服务依赖于已经废弃的函数调用（即 `function_call`），我们建议你迁移至工具调用（即 `tool_calls`），工具调用拓展了函数调用的能力，同时支持函数并行调用，关于工具调用的具体示例，可以参考我们的工具调用指南：

使用 Kimi API 完成工具调用（`tool_calls`）

下面是一个从 `functions` 迁移至 `tools` 的示例：

我们会将需要改造的部分代码以注释的形式呈现，并附上说明，以便于开发者能更好理解如何进行迁移。

python node.js

KIMI 开放平台

```
import httpx
from openai import OpenAI

client = OpenAI(
    api_key="MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台
    base_url="https://api.moonshot.cn/v1",
)

functions = [
    {
        "name": "search", # 函数的名称，请使用英文大小写字母、数据加上减号和下划线作为函数名
        "description": """
            通过搜索引擎搜索互联网上的内容。

            当你的知识无法回答用户提出的问题，或用户请求你进行联网搜索时，调用此工具。请从与用户
            搜索结果包含网站的标题、网站的地址（URL）以及网站简介。
        """, # 函数的介绍，在这里写上函数的具体作用以及使用场景，以便 Kimi 大模型能正确地选择
        "parameters": { # 使用 parameters 字段来定义函数接收的参数
            "type": "object", # 固定使用 type: object 来使 Kimi 大模型生成一个 JSON O
            "required": ["query"], # 使用 required 字段告诉 Kimi 大模型哪些参数是必填项
            "properties": { # properties 中是具体的参数定义，你可以定义多个参数
                "query": { # 在这里，key 是参数名称，value 是参数的具体定义
                    "type": "string", # 使用 type 定义参数类型
                    "description": """
                        用户搜索的内容，请从用户的提问或聊天上下文中提取。
                    """ # 使用 description 描述参数以便 Kimi 大模型更好地生成参数
                }
            }
        }
    }
]
```

```
def search_impl(query: str) -> List[Dict[str, Any]]:
    """
    search_impl 使用搜索引擎对 query 进行搜索，目前主流的搜索引擎（例如 Bing）都提供了 API
    你喜欢的搜索引擎 API 进行调用，并将返回结果中的网站标题、网站链接、网站简介信息放置在一个 d:
```

KIMI 开放平台



```
"""
r = httpx.get("https://your.search.api", params={"query": query})
return r.json()
>
```

```
def search(arguments: Dict[str, Any]) -> Any:
    query = arguments["query"]
    result = search_impl(query)
    return {"result": result}
```

```
function_map = {
    "search": search,
}
```

```
# =====
# tools 是 functions 的超集，因此我们可以通过已经定义的 functions 构造 tools，我们循环遍历
# 同时，我们也生成相应的 tool_map。
# =====
```

```
tools = []
tool_map = {}
for function in functions:
    tool = {
        "type": "function",
        "function": function,
    }
    tools.append(tool)
    tool_map[function["name"]] = function_map[function["name"]]
```

```
messages = [
    {"role": "system",
     "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你更擅长中文和英文的对话。你"},
    {"role": "user", "content": "请联网搜索 Context Caching，并告诉我它是什么。"} # 在
]
```

```
finish_reason = None
```

KIMI 开放平台



```
# while finish_reason is None or finish_reason == "function_call":
while finish_reason is None or finish_reason == "tool_calls":
    completion = client.chat.completions.create(
        model="kimi-k2.6",
        messages=messages,
        # =====
        # 我们弃用 functions 参数，而是使用 tools 参数来启用工具调用
        # =====
        # function=functions,
        tools=tools, # <-- 我们通过 tools 参数，将定义好的 tools 提交给 Kimi 大模型
    )
    choice = completion.choices[0]
    finish_reason = choice.finish_reason

# =====
# 在这里，我们将原先 function_call 的执行逻辑替换成 tool_calls 的执行逻辑；
# 注意，由于 tool_calls 可能有多个，因此我们需要通过 for 循环逐个执行每个 tool_call。
# =====
# if finish_reason == "function_call":
#     messages.append(choice.message)
#     function_call_name = choice.message.function_call.name
#     function_call_arguments = json.loads(choice.message.function_call.arguments)
#     function_call = function_map[function_call_name]
#     function_result = function_call(function_call_arguments)
#     messages.append({
#         "role": "function",
#         "name": function_call_name,
#         "content": json.dumps(function_result)
#     })

if finish_reason == "tool_calls": # <-- 判断当前返回内容是否包含 tool_calls
    messages.append(choice.message) # <-- 我们将 Kimi 大模型返回给我们的 assistant
    for tool_call in choice.message.tool_calls: # <-- tool_calls 可能是多个，因此
        tool_call_name = tool_call.function.name
        tool_call_arguments = json.loads(tool_call.function.arguments) # <--
        tool_function = tool_map[tool_call_name] # <-- 通过 tool_map 快速找到需
        tool_result = tool_function(tool_call_arguments)
```

KIMI

开放平台

注意，我们需要在 message 中提供 tool_call_id 和 name 字段，以便 Kimi 大模型
能正确匹配到对应的 tool_call。

```
messages.append({  
    >     "role": "tool",  
     "tool_call_id": tool_call.id,  
     "name": tool_call_name,  
     "content": json.dumps(tool_result), # <-- 我们约定使用字符串格式向 K  
})
```

```
print(choice.message.content) # <-- 在这里，我们才将模型生成的回复返回给用户
```

关于 tool_choice

Kimi API 支持 `tool_choice` 参数，但关于 `tool_choice` 参数的值与 OpenAI 有一些细微的差别。当前 Kimi API 与 OpenAI API 兼容的 `tool_choice` 值为：

- "none"
- "auto"
- null

请注意，当前版本的 Kimi API 暂时不支持 `tool_choice=required` 参数。

迁移建议：假如你的应用程序或服务依赖于 OpenAI API 中 `tool_choice` 字段的 `required` 值来确保大模型“一定”选择某个工具进行调用，我们建议使用一些特殊的手段来强化 Kimi 大模型对调用工具的认知以一定程度上兼容原有的业务逻辑，例如通过在提示词 prompt 中强调使用某个工具来达到类似的效果，我们通过简化版的代码来展示这一逻辑：

python node.js

KIMI 开放平台



```
def chat_completion(
    api_key="MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台
    base_url="https://api.moonshot.cn/v1",
)

tools = {
    # 在这里定义你的 tools
}

messages = [
    # 在这里存储你的消息历史记录
]

completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=messages,
    tools=tools,
    # tool_choice="required", # <-- 由于 Kimi API 暂时不支持 tool_choice=required,
)

choice = completion.choices[0]
if choice.finish_reason != "tool_calls":
    # 我们假定我们的业务逻辑能够确认此处必须要调用 tool_calls,
    # 在不使用 tool_choice=required 的情况下，我们通过提示词
    # prompt 来让 Kimi 强制选择一个工具进行调用
    messages.append(choice.message)
    messages.append({
        "role": "user",
        "content": "请选择一个工具 (tool) 来处理当前的问题。", # 通常情况下，Kimi 大模型能
    })
    completion = client.chat.completions.create(
        model="kimi-k2.6",
        messages=messages,
        tools=tools,
    )
    choice = completion.choices[0]
```



需要注意的是，这种方式并不能保证百分之百成功触发 `tool_calls`，如果你的应用程序或服务对 `tool_calls` 的调用有非常非常强的依赖，请等待 Kimi API 的 `tool_choice=required` 特性上线。

此页面对您有帮助吗？

 是

 否

[◀ Kimi CLI 使用](#)

